ASSIGNMENT 2

23CCE215 – MACHINE LEARNING

Name & Roll no.:

Jothsna Kishore - CB.EN.U4CCE24119

Hamshitha P - CB.EN.U4CCE24115

```
% Loading dataset
data = readtable('energydata_complete.csv');

% Display the size of the dataset
disp(size(data));
```

```
    19735       29
```

```
% Display the first few rows of the dataset
disp(head(data));
```

```
        date          Appliances    lights    T1      RH_1     T2      RH_2     T3      RH_3
T4      RH_4      T5      RH_5      T6      RH_6      T7      RH_7      T8      RH_8      T9
RH_9    T_out    Press_mm_hg    RH_out    Windspeed    Visibility    Tdewpoint    rv1
rv2
    ____________________    ___________    ______    ______    ______    ____    ______
_____    ______    ______    ______    ______    ______    ______    ______    ______
_____    ____    ______    ______    _____    ______    ____________    ______    _________
_________    _________    ______    ______

    2016-01-11 17:00:00       60        30       19.89    47.597    19.2    44.79    19.79    44.73
19    45.567    17.167     55.2    7.0267    84.257     17.2    41.627    18.2     48.9    17.033
45.53       6.6       733.5          92          7           63          5.3      13.275    13.275

    2016-01-11 17:10:00       60        30       19.89    46.693    19.2    44.722    19.79    44.79
19    45.992    17.167     55.2    6.8333    84.063     17.2     41.56    18.2    48.863    17.067
45.56    6.4833       733.6          92      6.6667       59.167          5.2      18.606    18.606

    2016-01-11 17:20:00       50        30       19.89     46.3    19.2    44.627    19.79    44.933
18.927     45.89    17.167     55.09     6.56    83.157     17.2    41.433    18.2     48.73       17
45.5    6.3667       733.7          92      6.3333       55.333          5.1      28.643    28.643
```

2016-01-11 17:30:00 50 40 19.89 46.067 19.2 44.59 19.79 45 18.89 45.723 17.167 55.09 6.4333 83.423 17.133 41.29 18.1 48.59 17 45.4 6.25 733.8 92 6 51.5 5 45.41 45.41

2016-01-11 17:40:00 60 40 19.89 46.333 19.2 44.53 19.79 45 18.89 45.53 17.2 55.09 6.3667 84.893 17.2 41.23 18.1 48.59 17 45.4 6.1333 733.9 92 5.6667 47.667 4.9 10.084 10.084

2016-01-11 17:50:00 50 40 19.89 46.027 19.2 44.5 19.79 44.933 18.89 45.73 17.133 55.03 6.3 85.767 17.133 41.26 18.1 48.59 17 45.29 6.0167 734 92 5.3333 43.833 4.8 44.919 44.919

2016-01-11 18:00:00 60 50 19.89 45.767 19.2 44.5 19.79 44.9 18.89 45.79 17.1 54.967 6.2633 86.09 17.133 41.2 18.1 48.59 17 45.29 5.9 734.1 92 5 40 4.7 47.234 47.234

2016-01-11 18:10:00 60 50 19.857 45.56 19.2 44.5 19.73 44.9 18.89 45.863 17.1 54.9 6.19 86.423 17.1 41.2 18.1 48.59 17 45.29 5.9167 734.17 91.833 5.1667 40 4.6833 33.04 33.04

```
% Includes statistics like mean, min, max for numeric columns and counts for categorical columns
summary(data);
```

data: 19735×29 table

Variables:

date: datetime

Appliances: double

lights: double

T1: double

RH_1: double

T2: double

RH_2: double

T3: double

RH_3: double

T4: double

RH_4: double

T5: double

RH_5: double

T6: double

RH_6: double

T7: double

RH_7: double

T8: double

RH_8: double

T9: double

RH_9: double

T_out: double

Press_mm_hg: double

RH_out: double

Windspeed: double

Visibility: double

Tdewpoint: double

rv1: double

rv2: double

Statistics for applicable variables:

	Num	Missing	Min	Median	Max	Mean	Std
date	0		2016-01-11 17:00:00	2016-03-20 05:30:00	2016-05-27 18:00:00	2016-03-20 05:30:00	949:31:28
Appliances	0		10	60	1080	97.6950	102.5249
lights	0		0	0	70	3.8019	7.9360
T1	0		16.7900	21.6000	26.2600	21.6866	1.6061

RH_1	0	27.0233	39.6567	63.3600	40.2597	3.9793
T2	0	16.1000	20	29.8567	20.3412	2.1930
RH_2	0	20.4633	40.5000	56.0267	40.4204	4.0698
T3	0	17.2000	22.1000	29.2360	22.2676	2.0061
RH_3	0	28.7667	38.5300	50.1633	39.2425	3.2546
T4	0	15.1000	20.6667	26.2000	20.8553	2.0429
RH_4	0	27.6600	38.4000	51.0900	39.0269	4.3413
T5	0	15.3300	19.3900	25.7950	19.5921	1.8446
RH_5	0	29.8150	49.0900	96.3217	50.9493	9.0220
T6	0	-6.0650	7.3000	28.2900	7.9109	6.0903
RH_6	0	1	55.2900	99.9000	54.6091	31.1498
T7	0	15.3900	20.0333	26	20.2671	2.1100
RH_7	0	23.2000	34.8633	51.4000	35.3882	5.1142
T8	0	16.3067	22.1000	27.2300	22.0291	1.9562
RH_8	0	29.6000	42.3750	58.7800	42.9362	5.2244
T9	0	14.8900	19.3900	24.5000	19.4858	2.0147
RH_9	0	29.1667	40.9000	53.3267	41.5524	4.1515
T_out	0	-5	6.9167	26.1000	7.4117	5.3174

Press_mm_hg	0	729.3000	756.1000	772.3000	755.5226	7.3994
RH_out	0	24	83.6667	100	79.7504	14.9011
Windspeed	0	0	3.6667	14	4.0398	2.4512
Visibility	0	1	40	66	38.3308	11.7947
Tdewpoint	0	-6.6000	3.4333	15.5000	3.7607	4.1946
rv1	0	0.0053	24.8977	49.9965	24.9880	14.4966
rv2	0	0.0053	24.8977	49.9965	24.9880	14.4966

```
% Reason: Raw timestamp is an identifier, but time patterns are useful.
data.date = datetime(data.date,'InputFormat','yyyy-MM-dd HH:mm:ss');

data.Hour  = hour(data.date);
data.Day   = day(data.date);
data.Month = month(data.date);

data.date = [];   % drop original date
disp(head(data));
```

Appliances	**lights**	**T1**	**RH_1**	**T2**	**RH_2**	**T3**	**RH_3**	**T4**	**RH_4**	**T5**	**RH_5**	**T6**	**RH_6**	**T7**	**RH_7**	**T8**	**RH_8**	**T9**	**RH_9**	**T_out**	**Press_mm_hg**	**RH_out**	**Windspeed**	**Visibility**	**Tdewpoint**	**rv1**	**rv2**	**Hour**	**Day**	**Month**
60	30	19.89	47.597	19.2	44.79	19.79	44.73	19	45.567	17.167	55.2	7.0267	84.257	17.2	41.627	18.2	48.9	17.033	45.53	6.6	733.5	92	7	63	5.3	13.275	13.275	17	11	1
60	30	19.89	46.693	19.2	44.722	19.79	44.79	19	45.992	17.167	55.2	6.8333	84.063	17.2	41.56	18.2	48.863	17.067	45.56	6.4833	733.6	92	6.6667	59.167	5.2	18.606	18.606	17	11	1

```
    50     30    19.89    46.3    19.2   44.627   19.79   44.933   18.927    45.89
17.167   55.09    6.56  83.157    17.2   41.433    18.2   48.73      17    45.5   6.3667
733.7     92   6.3333    55.333     5.1   28.643   28.643    17    11     1

    50     40    19.89   46.067   19.2    44.59   19.79      45   18.89   45.723
17.167   55.09   6.4333   83.423  17.133   41.29    18.1   48.59      17    45.4    6.25
733.8     92        6     51.5       5    45.41   45.41    17    11     1

    60     40    19.89   46.333   19.2    44.53   19.79      45   18.89    45.53    17.2
55.09   6.3667   84.893    17.2   41.23    18.1   48.59      17    45.4   6.1333    733.9
92    5.6667    47.667     4.9   10.084   10.084    17    11     1

    50     40    19.89   46.027   19.2     44.5   19.79   44.933   18.89    45.73
17.133   55.03     6.3   85.767  17.133   41.26    18.1   48.59      17   45.29   6.0167
734      92    5.3333    43.833     4.8   44.919   44.919    17    11     1

    60     50    19.89   45.767   19.2     44.5   19.79    44.9   18.89    45.79    17.1
54.967   6.2633   86.09  17.133    41.2    18.1   48.59      17   45.29     5.9    734.1
92       5        40       4.7   47.234   47.234    18    11     1

    60     50   19.857   45.56    19.2     44.5   19.73    44.9   18.89   45.863    17.1
54.9    6.19   86.423    17.1    41.2    18.1   48.59      17   45.29   5.9167    734.17
91.833   5.1667      40   4.6833   33.04    33.04    18    11     1
```

```
% Number of missing values
totalNaNs = sum(ismissing(data),'all');
fprintf('No. of missing values: %d',totalNaNs);
```

No. of missing values: 0

```
% Checking if duplicate rows are there
hasDuplicates = height(data) ~= height(unique(data,'rows'));

if hasDuplicates
    disp('Duplicate rows are present.');
else
    disp('No duplicate rows are present.');
end
```

No duplicate rows are present.

```
% Select numeric columns
numVars = varfun(@isnumeric, data, 'OutputFormat','uniform');
numData = data{:, numVars};
featureNames = data.Properties.VariableNames(numVars);
outlierCount = zeros(length(featureNames),1);

lowerBound = zeros(1,length(featureNames));
upperBound = zeros(1,length(featureNames));

for i = 1:length(featureNames)
```

```
    col = numData(:,i);

    % IQR calculation
    Q1 = prctile(col,25);
    Q3 = prctile(col,75);
    IQR = Q3 - Q1;

    % If IQR is zero, skip
    if IQR == 0
        outlierCount(i) = 0;
        lowerBound(i) = Q1;
        upperBound(i) = Q3;
        continue
    end

    % Define the lower threshold for outliers
    lowerBound(i) = Q1 - 1.5*IQR;
    % Define the upper threshold for outliers
    upperBound(i) = Q3 + 1.5*IQR;

    outlierCount(i) = sum(col < lowerBound(i) | col > upperBound(i));
end

 outlierTable = table(featureNames', outlierCount, 'VariableNames',
{'Feature','OutlierCount'});
 disp(outlierTable);
```

Feature	OutlierCount
{'Appliances' }	2138
{'lights' }	0
{'T1' }	515
{'RH_1' }	146
{'T2' }	546
{'RH_2' }	235
{'T3' }	217
{'RH_3' }	15
{'T4' }	186
{'RH_4' }	0

{'T5' } 172

{'RH_5' } 1330

{'T6' } 515

{'RH_6' } 0

{'T7' } 2

{'RH_7' } 42

{'T8' } 71

{'RH_8' } 17

{'T9' } 0

{'RH_9' } 21

{'T_out' } 436

{'Press_mm_hg'} 219

{'RH_out' } 239

{'Windspeed' } 214

{'Visibility' } 2522

{'Tdewpoint' } 10

{'rv1' } 0

{'rv2' } 0

{'Hour' } 0

{'Day' } 0

{'Month' } 0

```
totalOutliersBefore = sum(outlierCount);
fprintf('Total number of outliers before handling: %d\n', totalOutliersBefore);
```

Total number of outliers before handling: 9808

```
% Handling outliers

% Replace values in X that are below lowerBound with lowerBound
numData = bsxfun(@max,numData, lowerBound);

% Replace values in X that are above upperBound with upperBound
numData = bsxfun(@min,numData, upperBound);

% Update the original dataset with the outlier-handled numeric features
data{:,numVars} = numData;
```

```
totalOutliersAfter = 0;
fprintf('Total number of outliers after handling: %d\n', totalOutliersAfter);
```

Total number of outliers after handling: 0

```
disp("Dataset after outlier handling:");
```

Dataset after outlier handling:

```
disp(head(data));
```

Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3	T4	RH_4	T5	RH_5	T6	RH_6	T7	RH_7	T8	RH_8	T9	RH_9	T_out	Press_mm_hg	RH_out	Windspeed	Visibility	Tdewpoint	rv1	rv2	Hour	Day	Month
60	0	19.89	47.597	19.2	44.79	19.79	44.73	19	45.567	17.167	55.2	7.0267	84.257	17.2	41.627	18.2	48.9	17.033	45.53	6.6	735.93	92	7	56.5	5.3	13.275	13.275	17	11	1
60	0	19.89	46.693	19.2	44.722	19.79	44.79	19	45.992	17.167	55.2	6.8333	84.063	17.2	41.56	18.2	48.863	17.067	45.56	6.4833	735.93	92	6.6667	56.5	5.2	18.606	18.606	17	11	1
50	0	19.89	46.3	19.2	44.627	19.79	44.933	18.927	45.89	17.167	55.09	6.56	83.157	17.2	41.433	18.2	48.73	17	45.5	6.3667	735.93	92	6.3333	55.333	5.1	28.643	28.643	17	11	1
50	0	19.89	46.067	19.2	44.59	19.79	45	18.89	45.723	17.167	55.09	6.4333	83.423	17.133	41.29	18.1	48.59	17	45.4	6.25	735.93	92	6	51.5	5	45.41	45.41	17	11	1
60	0	19.89	46.333	19.2	44.53	19.79	45	18.89	45.53	17.2	55.09	6.3667	84.893	17.2	41.23	18.1	48.59	17	45.4	6.1333	735.93	92	5.6667	47.667	4.9	10.084	10.084	17	11	1
50	0	19.89	46.027	19.2	44.5	19.79	44.933	18.89	45.73	17.133	55.03	6.3	85.767	17.133	41.26	18.1	48.59	17	45.29	6.0167	735.93	92	5.3333	43.833	4.8	44.919	44.919	17	11	1
60	0	19.89	45.767	19.2	44.5	19.79	44.9	18.89	45.79	17.1	54.967	6.2633	86.09	17.133	41.2	18.1	48.59	17	45.29	5.9	735.93	92	5	40	4.7	47.234	47.234	18	11	1

```
      60         0      19.857     45.56    19.2      44.5    19.73      44.9     18.89    45.863      17.1
54.9      6.19    86.423      17.1      41.2    18.1     48.59        17    45.29    5.9167      735.93
91.833     5.1667          40       4.6833       33.04     33.04     18     11       1
```

```
% Extract the target variable 'Appliances' from the dataset and store it in y
% This is the variable we want to predict
y = data.Appliances;

% Copy the entire dataset to X, which will be used as the input/features
X = data;

% Remove the target column from X to ensure it only contains predictor features
X.Appliances = [];
% Correlation study

% Identify numeric features in X
numVarsX = varfun(@isnumeric, X, 'OutputFormat','uniform');

% Compute the correlation of each numeric feature with the target variable 'y'
corrValues = corr(X{:,numVarsX}, y);

% Create a table combining feature names and their correlation with the target
corrTable = table( ...
    X.Properties.VariableNames(numVarsX)', ...
    corrValues, ...
    'VariableNames', {'Feature','CorrelationWithAppliances'});

disp(corrTable)
```

Feature	CorrelationWithAppliances
{'lights'}	NaN
{'T1'}	0.14627
{'RH_1'}	0.0915
{'T2'}	0.20964
{'RH_2'}	-0.095974
{'T3'}	0.14557
{'RH_3'}	0.0047197
{'T4'}	0.11318
{'RH_4'}	0.0004889
{'T5'}	0.08955

{'RH_5' } -0.0060199

{'T6' } 0.19367

{'RH_6' } -0.16316

{'T7' } 0.090106

{'RH_7' } -0.096975

{'T8' } 0.13603

{'RH_8' } -0.16679

{'T9' } 0.070581

{'RH_9' } -0.12058

{'T_out' } 0.17254

{'Press_mm_hg'} -0.065848

{'RH_out' } -0.23253

{'Windspeed' } 0.098671

{'Visibility' } -0.0042465

{'Tdewpoint' } 0.048584

{'rv1' } -0.0083627

{'rv2' } -0.0083627

{'Hour' } 0.3511

{'Day' } -0.0099368

{'Month' } 0.044723

```
corrValues(isnan(corrValues)) = 0;

%{
0.05 is chosen because it represents a very weak
correlation. Features below this value are
considered almost uncorrelated with the target,
so keeping them adds little predictive value
and may introduce noise.
%}
threshold = 0.05;

%Identifies index of features with absolute low correlation with target
lowCorrIdx = abs(corrValues) < threshold;
lowCorrFeatures = corrTable.Feature(lowCorrIdx);

disp("Dropped due to low correlation:")
```

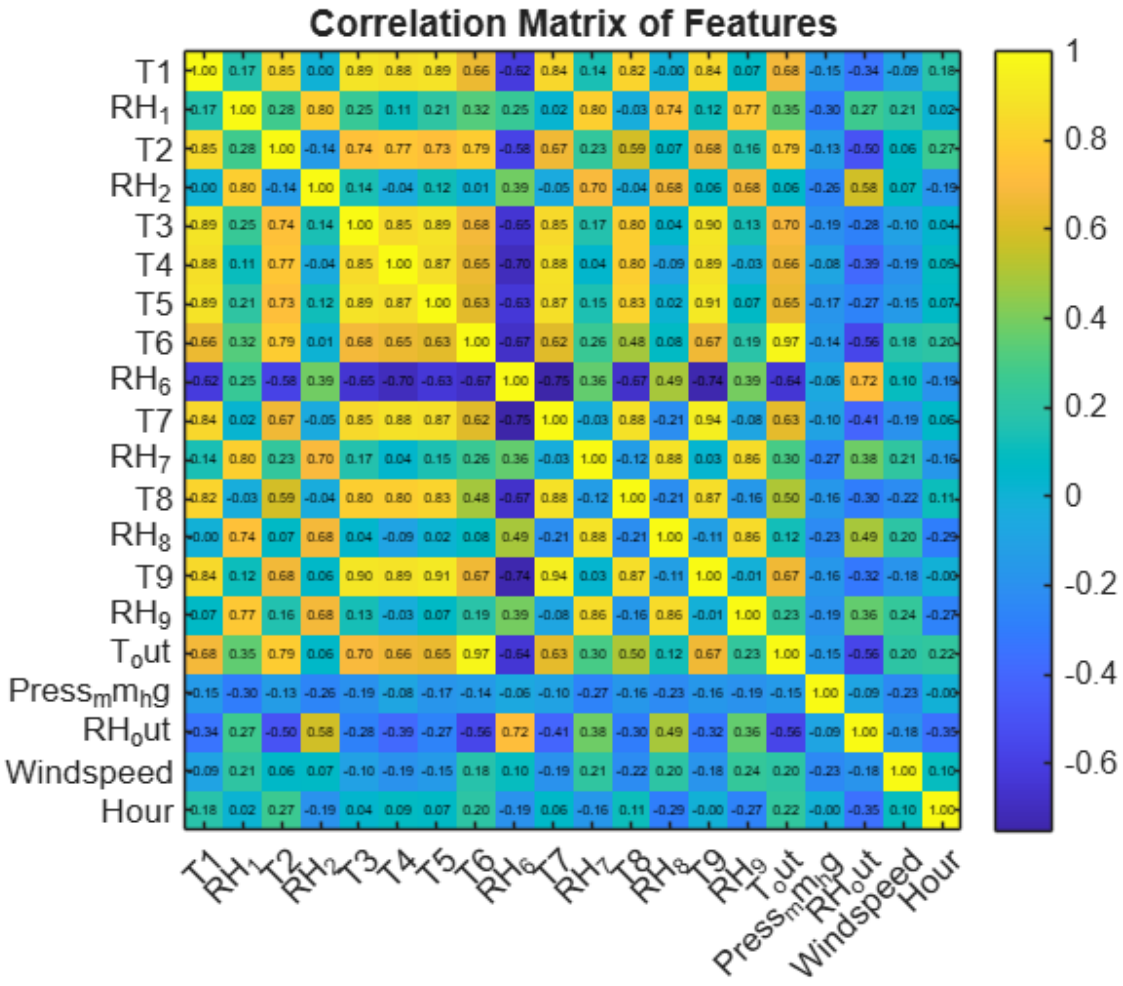
Dropped due to low correlation:

```
disp(lowCorrFeatures)
```

```
{'lights'    }
{'RH_3'      }
{'RH_4'      }
{'RH_5'      }
{'Visibility'}
{'Tdewpoint' }
{'rv1'       }
{'rv2'       }
{'Day'       }
{'Month'     }
```

```
% Remove low correlated features
X(:, lowCorrFeatures) = [];

%{
rv1 and rv2 are randomly generated variables added to test
whether a regression model can identify and ignore noise stated
by original UCI Energy Efficiency dataset description.
%}
% Identify which columns in X are numeric
% Returns a logical array: true for numeric columns, false otherwise
numVarsX = varfun(@isnumeric, X, 'OutputFormat','uniform');

% Extract the names of numeric features for labeling purposes
featureNames = X.Properties.VariableNames(numVarsX);

% Compute the correlation matrix for all numeric features
corrMatrix = corr(X{:, numVarsX});

% Plot the correlation matrix as a heatmap
figure
imagesc(corrMatrix)
colorbar
axis equal tight

xticks(1:numel(featureNames))
yticks(1:numel(featureNames))
xticklabels(featureNames)
yticklabels(featureNames)
```

```
xtickangle(45)

% Write values inside cells
for i = 1:size(corrMatrix,1)
    for j = 1:size(corrMatrix,2)
        text(j, i, sprintf('%.2f', corrMatrix(i,j)), ...
            'HorizontalAlignment','center', ...
            'Color','k', 'FontSize',4);
    end
end

title('Correlation Matrix of Features')
```




```
% Identifies highly correlated features
highCorr = abs(corrMatrix) > 0.9;

% Set the diagonal elements to 0 because a feature
% is always perfectly correlated with itself
highCorr(eye(size(highCorr)) == 1) = 0;
```

```
% Identify redundant feature indices
[rows, cols] = find(triu(highCorr,1));

% Choose second feature from each correlated pair for removal
highCorrFeatures = unique(featureNames(cols));

disp("Dropped due to high inter-feature correlation:")
```

Dropped due to high inter-feature correlation:

```
disp(highCorrFeatures)
```

```
    {'T9'}    {'T_out'}
```

```
% Remove highly correlated features
X(:, highCorrFeatures) = [];

%{
Highly correlated features were removed to avoid
multicollinearity in Multiple Linear Regression.
Multicollinearity leads to unstable regression coefficients,
inflated standard errors, and unreliable interpretation.
Removing redundant predictors improves numerical stability and
model generalization without significant loss of information.
%}
% Dataframe after correlation study
disp("Dataframe after dropping features:")
```

Dataframe after dropping features:

```
disp(head(X))
```

T1	RH_1	T2	RH_2	T3	T4	T5	T6	RH_6	T7	RH_7	T8	RH_8	RH_9	Press_mm_hg	RH_out	Windspeed	Hour
19.89	47.597	19.2	44.79	19.79	19	17.167	7.0267	84.257	17.2	41.627	18.2	48.9	45.53	735.93	92	7	17
19.89	46.693	19.2	44.722	19.79	19	17.167	6.8333	84.063	17.2	41.56	18.2	48.863	45.56	735.93	92	6.6667	17
19.89	46.3	19.2	44.627	19.79	18.927	17.167	6.56	83.157	17.2	41.433	18.2	48.73	45.5	735.93	92	6.3333	17
19.89	46.067	19.2	44.59	19.79	18.89	17.167	6.4333	83.423	17.133	41.29	18.1	48.59	45.4	735.93	92	6	17

19.89 46.333 19.2 44.53 19.79 18.89 17.2 6.3667 84.893 17.2
41.23 18.1 48.59 45.4 735.93 92 5.6667 17

19.89 46.027 19.2 44.5 19.79 18.89 17.133 6.3 85.767 17.133
41.26 18.1 48.59 45.29 735.93 92 5.3333 17

19.89 45.767 19.2 44.5 19.79 18.89 17.1 6.2633 86.09 17.133 41.2
18.1 48.59 45.29 735.93 92 5 18

19.857 45.56 19.2 44.5 19.73 18.89 17.1 6.19 86.423 17.1 41.2
18.1 48.59 45.29 735.93 91.833 5.1667 18

```
% Normalising input variables
X = normalize(X);
disp(head(X))
```

T1	RH_1	T2	RH_2	T3	T4	T5	T6	RH_6	T7
RH_7	T8	RH_8	RH_9	Press_mm_hg	RH_out	Windspeed	Hour		
-1.139	1.8634	-0.5287	1.0926	-1.2451	-0.91261	-1.319	-0.13944	0.95177	
-1.4536	1.2209	-1.961	1.142	0.95827	-2.6842	0.82888	1.2262	0.79428	
-1.139	1.6343	-0.5287	1.0756	-1.2451	-0.91261	-1.319	-0.172	0.94557	
-1.4536	1.2078	-1.961	1.135	0.9655	-2.6842	0.82888	1.0886	0.79428	
-1.139	1.5345	-0.5287	1.0515	-1.2451	-0.94864	-1.319	-0.21804	0.91646	
-1.4536	1.1831	-1.961	1.1095	0.95104	-2.6842	0.82888	0.95094	0.79428	
-1.139	1.4754	-0.5287	1.0423	-1.2451	-0.96665	-1.319	-0.23938	0.92502	
-1.4852	1.155	-2.0122	1.0826	0.92695	-2.6842	0.82888	0.81331	0.79428	
-1.139	1.543	-0.5287	1.0273	-1.2451	-0.96665	-1.3009	-0.25061	0.97221	
-1.4536	1.1433	-2.0122	1.0826	0.92695	-2.6842	0.82888	0.67568	0.79428	
-1.139	1.4652	-0.5287	1.0197	-1.2451	-0.96665	-1.3372	-0.26184	1.0002	
-1.4852	1.1491	-2.0122	1.0826	0.90045	-2.6842	0.82888	0.53805	0.79428	
-1.139	1.3993	-0.5287	1.0197	-1.2451	-0.96665	-1.3553	-0.26802	1.0106	
-1.4852	1.1374	-2.0122	1.0826	0.90045	-2.6842	0.82888	0.40042	0.93875	
-1.1601	1.3468	-0.5287	1.0197	-1.2754	-0.96665	-1.3553	-0.28037	1.0213	
-1.501	1.1374	-2.0122	1.0826	0.90045	-2.6842	0.81755	0.46923	0.93875	

```
% Combine final dataset
finalData = [X table(y,'VariableNames',{'Appliances'})];
```

```
disp("Final dataframe:")
```

Final dataframe:

```
disp(head(finalData))
```

T1	RH_1	T2	RH_2	T3	T4	T5	T6	RH_6	T7	RH_7	T8	RH_8	RH_9	Press_mm_hg	RH_out	Windspeed	Hour	Appliances
-1.139	1.8634	-0.5287	1.0926	-1.2451	-0.91261	-1.319	-0.13944	0.95177	-1.4536	1.2209	-1.961	1.142	0.95827	-2.6842	0.82888	1.2262	0.79428	60
-1.139	1.6343	-0.5287	1.0756	-1.2451	-0.91261	-1.319	-0.172	0.94557	-1.4536	1.2078	-1.961	1.135	0.9655	-2.6842	0.82888	1.0886	0.79428	60
-1.139	1.5345	-0.5287	1.0515	-1.2451	-0.94864	-1.319	-0.21804	0.91646	-1.4536	1.1831	-1.961	1.1095	0.95104	-2.6842	0.82888	0.95094	0.79428	50
-1.139	1.4754	-0.5287	1.0423	-1.2451	-0.96665	-1.319	-0.23938	0.92502	-1.4852	1.155	-2.0122	1.0826	0.92695	-2.6842	0.82888	0.81331	0.79428	50
-1.139	1.543	-0.5287	1.0273	-1.2451	-0.96665	-1.3009	-0.25061	0.97221	-1.4536	1.1433	-2.0122	1.0826	0.92695	-2.6842	0.82888	0.67568	0.79428	60
-1.139	1.4652	-0.5287	1.0197	-1.2451	-0.96665	-1.3372	-0.26184	1.0002	-1.4852	1.1491	-2.0122	1.0826	0.90045	-2.6842	0.82888	0.53805	0.79428	50
-1.139	1.3993	-0.5287	1.0197	-1.2451	-0.96665	-1.3553	-0.26802	1.0106	-1.4852	1.1374	-2.0122	1.0826	0.90045	-2.6842	0.82888	0.40042	0.93875	60
-1.1601	1.3468	-0.5287	1.0197	-1.2754	-0.96665	-1.3553	-0.28037	1.0213	-1.501	1.1374	-2.0122	1.0826	0.90045	-2.6842	0.81755	0.46923	0.93875	60

```
% Set random seed for reproducibility so that the train-test split is consistent every time
rng(1);
```

```
% Create a partition object for hold-out cross-validation
% 'HoldOut',0.2 means 20% of the data will be used as the test set, 80% as training
cv = cvpartition(height(finalData),'HoldOut',0.2);

% Select rows marked for training from the partition and store them in trainData
trainData = finalData(training(cv),:);
disp("Training set data:")
```

Training set data:

```
disp(head(trainData))
```

T1	RH_1	T2	RH_2	T3	T4	T5	T6	RH_6	T7	RH_7	T8	RH_8	RH_9	Press_mm_hg	RH_out	Windspeed	Hour	Appliances
-1.139	1.8634	-0.5287	1.0926	-1.2451	-0.91261	-1.319	-0.13944	0.95177	-1.4536	1.2209	-1.961	1.142	0.95827	-2.6842	0.82888	1.2262	0.79428	60
-1.139	1.6343	-0.5287	1.0756	-1.2451	-0.91261	-1.319	-0.172	0.94557	-1.4536	1.2078	-1.961	1.135	0.9655	-2.6842	0.82888	1.0886	0.79428	60
-1.139	1.5345	-0.5287	1.0515	-1.2451	-0.94864	-1.319	-0.21804	0.91646	-1.4536	1.1831	-1.961	1.1095	0.95104	-2.6842	0.82888	0.95094	0.79428	50
-1.139	1.4754	-0.5287	1.0423	-1.2451	-0.96665	-1.319	-0.23938	0.92502	-1.4852	1.155	-2.0122	1.0826	0.92695	-2.6842	0.82888	0.81331	0.79428	50
-1.139	1.543	-0.5287	1.0273	-1.2451	-0.96665	-1.3009	-0.25061	0.97221	-1.4536	1.1433	-2.0122	1.0826	0.92695	-2.6842	0.82888	0.67568	0.79428	60
-1.1601	1.3468	-0.5287	1.0197	-1.2754	-0.96665	-1.3553	-0.28037	1.0213	-1.501	1.1374	-2.0122	1.0826	0.90045	-2.6842	0.81755	0.46923	0.93875	60
-1.2023	1.3564	-0.5287	1.003	-1.2754	-0.96665	-1.319	-0.2916	1.0471	-1.4694	1.1765	-2.0122	1.0826	0.90045	-2.6842	0.80621	0.53805	0.93875	60

-1.1601 1.4813 -0.51438 0.99463 -1.2451 -0.96665 -1.3553 -0.28037 1.06 -1.4536 1.1961 -2.0122 1.0826 0.90045 -2.6842 0.79487 0.60686 0.93875 70

```
% Select rows marked for testing from the partition and store them in testData
testData  = finalData(test(cv),:);
disp("Test set data:")
```

Test set data:

```
disp(head(testData))
```

T1	RH_1	T2	RH_2	T3	T4	T5	T6	RH_6	T7	RH_7	T8	RH_8	RH_9	Press_mm_hg	RH_out	Windspeed	Hour	Appliances
-1.139	1.4652	-0.5287	1.0197	-1.2451	-0.96665	-1.3372	-0.26184	1.0002	-1.4852	1.1491	-2.0122	1.0826	0.90045	-2.6842	0.82888	0.53805	0.79428	50
-1.139	1.3993	-0.5287	1.0197	-1.2451	-0.96665	-1.3553	-0.26802	1.0106	-1.4852	1.1374	-2.0122	1.0826	0.90045	-2.6842	0.82888	0.40042	0.93875	60
-0.79943	2.8958	-0.21509	1.3545	-1.1224	-0.91261	-1.3553	-0.32922	1.077	-1.0918	1.9853	-2.0122	1.1982	0.9137	-2.6842	0.69282	0.81331	1.0832	100
-0.50622	2.0756	-0.051121	1.3267	-1.0552	-0.82664	-0.82898	-0.40221	1.0569	-1.264	1.3592	-1.6896	1.3967	0.86109	-2.6842	0.53408	0.81331	1.2277	120
-0.41551	1.7282	-0.0081388	1.2014	-1.0081	-0.75376	-1.0831	-0.38649	1.0258	-1.2482	1.6135	-1.6077	1.3839	0.84503	-2.6842	0.5114	0.81331	1.2277	110
-0.25309	1.4331	0.10807	1.1679	-0.94255	-0.72101	-0.67107	-0.39154	0.99907	-1.1266	1.6442	-1.4831	1.2824	0.77998	-2.6842	0.5114	0.81331	1.3722	100
-0.056909	1.0019	0.33095	0.79125	-0.92868	0.15183	-0.16286	-0.31799	0.9689	-1.2166	1.4426	-1.3688	1.2435	0.71011	-2.634	0.48872	1.2262	1.6611	60

-0.12019 1.0095 0.29592 0.83393 -0.99298 0.0036255 -0.16286 -0.37582
1.0268 -1.2166 1.5679 -1.3517 1.4413 0.93498 -2.593 0.6588 1.0198
1.6611 70

```
% Save preprocessed data
writetable(trainData,'train_preprocessed.csv');
writetable(testData,'test_preprocessed.csv');
% Load the preprocessed training and testing data
trainData = readtable('train_preprocessed.csv');
testData  = readtable('test_preprocessed.csv');
% Extract target variable (Appliances)
y_train = trainData.Appliances;
y_test  = testData.Appliances;

% Copy predictor variables
X_train = trainData;
X_test  = testData;

% Remove target column from predictors
X_train.Appliances = [];
X_test.Appliances  = [];
% Convert tables to matrices for regression computation
Xtrain_mat = X_train{:,:};
Xtest_mat  = X_test{:,:};
% Add a column of ones to include intercept term in regression
Xtrain_mat = [ones(size(Xtrain_mat,1),1) Xtrain_mat];
Xtest_mat  = [ones(size(Xtest_mat,1),1) Xtest_mat];
% Train model using Normal Equation
% beta = (X'X)^(-1) X'y
beta = (Xtrain_mat' * Xtrain_mat) \ (Xtrain_mat' * y_train);
% Predict appliance energy consumption
y_pred_train = Xtrain_mat * beta;
y_pred_test  = Xtest_mat * beta;
% Display regression coefficients with feature names
featureNames = ["Intercept", X_train.Properties.VariableNames];

coeffTable = table(featureNames', beta, ...
   'VariableNames', {'Feature','Coefficient'});

disp(coeffTable)
```

Feature	Coefficient
"Intercept"	79.2
"T1"	0.59207

"RH_1"	32.738
"T2"	-13.175
"RH_2"	-23.606
"T3"	17.08
"T4"	0.31787
"T5"	-3.3003
"T6"	4.3632
"RH_6"	7.4545
"T7"	-13.964
"RH_7"	0.7962
"T8"	13.126
"RH_8"	-14.582
"RH_9"	-3.3589
"Press_mm_hg"	0.47594
"RH_out"	-1.1305
"Windspeed"	2.9159
"Hour"	7.3197

```
% Combine train and test actuals and predictions
y_all = [y_train; y_test];
y_pred_all = [y_pred_train; y_pred_test];

% Mean Squared Error
MSE = mean((y_all - y_pred_all).^2);

% Root Mean Squared Error
RMSE = sqrt(MSE);

% Mean Absolute Error
MAE = mean(abs(y_all - y_pred_all));

% R-squared
SS_res = sum((y_all - y_pred_all).^2);
SS_tot = sum((y_all - mean(y_all)).^2);
R2 = 1 - (SS_res / SS_tot);
% Create table
EvaluationTable = table(MSE, RMSE, MAE, R2, ...
    'VariableNames', {'MSE','RMSE','MAE','R_squared'});
```

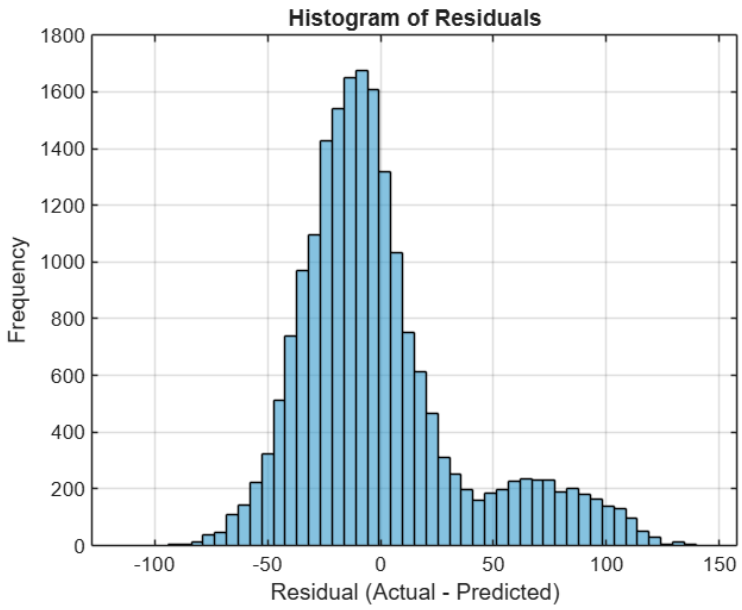
```
disp('Model Evaluation Metrics:')
```

Model Evaluation Metrics:

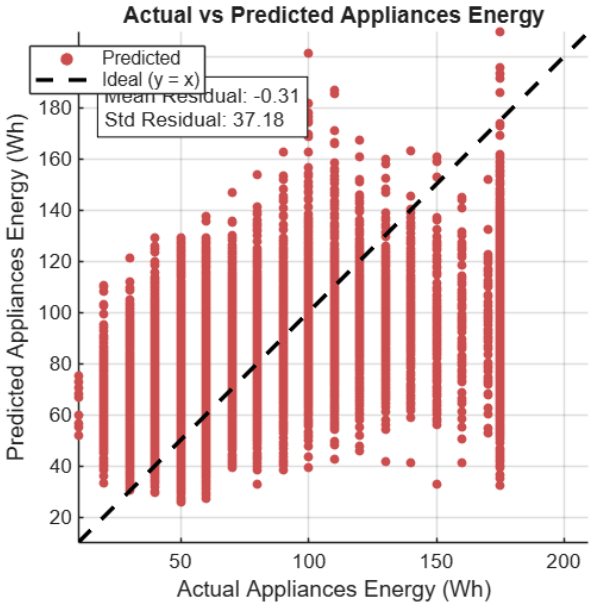
```
disp(EvaluationTable)
```

MSE	RMSE	MAE	R_squared
1382.5	37.182	27.519	0.2509

```
%{
The regression model achieved an RMSE of 37.18 Wh and an MAE of 27.52 Wh.
The R² value of 0.25 indicates that approximately 25% of the variance in
appliance energy consumption is explained by the linear model. This is
reasonable given the stochastic nature of human energy usage and the
limitations of linear modeling.
%}
% Model performance visualisation

% Compute residuals (errors)
residuals = y_all - y_pred_all;

% 1. Histogram of Residuals
figure('Color', 'w');
histogram(residuals, 50, 'FaceColor', [0.2 0.6 0.8], 'EdgeColor', 'k');
xlabel('Residual (Actual - Predicted)');
ylabel('Frequency');
title('Histogram of Residuals');
grid on;
```



```
% 2. Scatter plot: Actual vs Predicted
figure('Color', 'w');

scatter(y_all, y_pred_all, 20, 'filled', 'MarkerFaceColor', [0.8 0.3 0.3]);
hold on;

% Plot ideal line (y = x)
maxVal = max(max(y_all), max(y_pred_all));
minVal = min(min(y_all), min(y_pred_all));
plot([minVal, maxVal], [minVal, maxVal], 'k--', 'LineWidth', 2);

xlabel('Actual Appliances Energy (Wh)');
ylabel('Predicted Appliances Energy (Wh)');
title('Actual vs Predicted Appliances Energy');
legend('Predicted', 'Ideal (y = x)', 'Location', 'best');
grid on;
axis tight;
axis equal;

% Residual statistics on plot
res_mean = mean(residuals);
res_std  = std(residuals);

% Position text inside the axes based on relative values
x_text = minVal + 0.05*(maxVal - minVal);
y_text = minVal + 0.9*(maxVal - minVal);
text(x_text, y_text, ...
    sprintf('Mean Residual: %.2f\nStd Residual: %.2f', res_mean, res_std), ...
    'FontSize', 10, 'BackgroundColor', 'w', 'EdgeColor', 'k', ...
    'VerticalAlignment', 'top', 'HorizontalAlignment', 'left');
```



```
% RESULTS:
% ------------------------------------------------------------------------
% • The dataset was preprocessed successfully, with no missing values and
%   duplicate records removed where necessary.
% • Correlation analysis showed varying degrees of association between the
%   input features and appliance energy consumption, with no single feature
%   dominating the prediction.
% • The multiple linear regression model achieved an R² value of
%   approximately 0.25, indicating that about 25% of the variance in
%   appliance energy consumption is explained by the selected predictors.
% • The Actual vs Predicted scatter plot exhibits a positive trend along
%   the ideal y = x line, demonstrating that the model captures the general
%   linear relationship between the predictors and energy consumption,
%   although with noticeable dispersion.
% • The residual histogram is centered close to zero, suggesting the
%   absence of strong systematic bias, while the spread of residuals
%   indicates the presence of occasional larger prediction errors.

% CONCLUSION:
% ------------------------------------------------------------------------
% The obtained R² value of approximately 0.25 reflects moderate predictive
% performance of the multiple linear regression model. While the model does
% not fully explain appliance energy consumption, it captures meaningful
% linear trends present in the data.

% The results indicate that appliance energy usage is influenced not only
% by the measured environmental variables but also by additional complex
% and potentially non-linear factors such as occupant behavior and usage
% patterns, which are not fully captured by a linear model.

% Nevertheless, the observed regression trend demonstrates that
% machine learning-based prediction can support traditional energy analysis
% methods by reducing computational effort and providing useful baseline
% estimates of appliance energy consumption.
```